

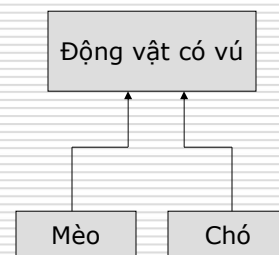
THỪA KẾ VÀ ĐA HÌNH

1

1

ĐẶC BIỆT HÓA & TỔNG QUÁT HÓA

- Lớp và các thể hiện của lớp (đối tượng) tuy không cùng tồn tại trong cùng một khối nhưng chúng tồn tại trong một mạng lưới các quan hệ phụ thuộc lẫn nhau.
- Quan hệ là một (is-a) là một sự đặc biệt hóa
 - Mèo là một loài động vật có vú. Nó có tất cả các đặc tính của một loài động vật có vú và có thêm các thuộc tính đặc trưng riêng của họ nhà mèo.
- Quan hệ đặc biệt hóa & tổng quát hóa là đối ngẫu và phân cấp
 - Tạo ra cây quan hệ
 - Di chuyển đi lên là tổng quát hóa, ngược lại di chuyển đi xuống là đặc biệt hóa.



2

2

KẾ THỪA

- ❑ Quan hệ đặc biệt hóa được thực thi bằng cách sử dụng sự **kế thừa**.
- ❑ Trong ngôn ngữ C#, để tạo một lớp kế thừa từ lớp khác, thêm dấu ":" vào sau tên lớp đó
 - `public class Meo : Dongvatcovu`
 - Lớp kế thừa (Meo) gọi là lớp dẫn xuất
 - Lớp được kế thừa (Dongvatcovu) gọi là lớp được dẫn xuất hoặc lớp cơ sở.
- ❑ Lớp dẫn xuất kế thừa tất cả các thành phần của lớp cơ sở.
- ❑ Lớp dẫn xuất có thể tạo phương thức mới bằng việc đánh dấu với từ khóa **new**

3

3

VÍ DỤ

```
class A {           // lớp cơ sở
    public int a;
    public A() {...}
    public void F() {...}
}

class B : A {       // lớp con (kế thừa từ A, mở rộng A)
    int b;
    public B() {...}
    public void G() {...}
}
```

- ❑ Lớp B kế thừa thuộc tính a và phương thức F, bổ sung thêm thuộc tính b và phương thức G
 - Hàm tạo không được kế thừa
 - Các phương thức được kế thừa có thể chồng lớp con
- ❑ Một lớp chỉ được kế thừa từ lớp khác nhưng có thể cài đặt cho nhiều interface.
- ❑ Một lớp chỉ có thể kế thừa từ một lớp khác không phải là struct
- ❑ Một lớp không có lớp cơ sở tường minh sẽ kế thừa từ lớp **object**

4

4

GÁN VÀ KIỂM TRA KIỂU

Khai báo

```
class A {...}
class B : A {...}
class C: B {...}
```

Lệnh gán

```
A a = new A(); // kiểu static của biến a là kiểu khi khai báo (A)
               // kiểu dynamic type của biến a là kiểu của đối tượng được khởi tạo (A)
a = new B();   // kiểu dynamic là B
a = new C();   // kiểu dynamic là C

B b = a;       // lỗi khi dịch chương trình
```

Kiểm tra kiểu khi thực thi (Toán tử is)

```
a = new C();
if (a is C) ... // true, nếu kiểu dynamic của a là C hoặc subclass của C
if (a is B) ... // true
if (a is A) ... // true
a = null;
if (a is C) ... // false
```

5

5

ÉP KIỂU

Ép kiểu

```
A a = new C();
B b = (B) a; // đúng, nếu a có kiểu dynamic là B hoặc subclass của B
C c = (C) a;

a = null;
c = (C) a; // đúng, c = null
```

Toán tử as

```
A a = new C();
B b = a as B; // if (a is B) b = (B)a; else b = null;
C c = a as C;

a = null;
c = a as C; // c == null
```

6

6

CHỒNG PHƯƠNG THỨC

Chỉ những phương thức được khai báo với từ khoá `virtual` mới được chồng.

```
class A {
    public void F() {...} // F không được chồng
    public virtual void G() {...} // Có thể chồng G trong subclass
}
```

Các phương thức chồng được khai báo với từ khoá **override**

```
class B : A {
    public void F() {...} // nên sử dụng từ khoá new
    public void G() {...} // nên sử dụng từ khoá new
    public override void G() { // ok: chồng toán tử G
        ... base.G(); // gọi G() của lớp cơ sở
    }
}
```

- ☐ Thuộc tính và indexer cũng được chồng.
- ☐ Phương thức static không được chồng

7

7

GÁN ĐỘNG

```
class A {
    public virtual void WhoAreYou() {Console.WriteLine("I am an A"); }
}
class B : A {
    public override void WhoAreYou() { Console.WriteLine("I am a B"); }
}
```

Thông điệp viện dẫn phương thức phụ thuộc vào kiểu dynamic của đối tượng.

```
A a = new B();
a.WhoAreYou(); // "I am a B"
```

Ưu điểm: mọi phương thức có thể làm việc với A đều làm việc được với B

```
void Use (A x) {
    x.WhoAreYou();
}

Use(new A()); // "I am an A"
Use(new B()); // "I am a B"
```

8

8

ẨN

Trong subclass, các thành phần được khai báo với từ khoá **new** sẽ ẩn tất cả các thành phần có cùng tên và dấu hiệu trong baseclass.

```
class A {
    public int x;
    public void F() {...}
    public virtual void G() {...}
}
class B : A {
    public new int x;
    public new void F() {...}
    public new void G() {...}
}

B b = new B();
b.x = ...;           // truy nhập tới B.x
b.F(); ... b.G();    // gọi tới B.F và B.G

((A)b).x = ...;      // truy nhập tới A.x
((A)b).F(); ... ((A)b).G(); // gọi tới A.F và A.G
```

9

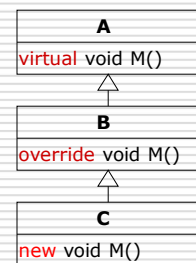
9

VÍ DỤ

```
class Animal {
    public virtual void WhoAreYou() {
        Console.WriteLine("I am an animal"); }
}
```

```
class Cat : Animal {
    public override void WhoAreYou() {
        Console.WriteLine("I am a cat"); }
}
```

```
Animal pet = new Cat();
pet.WhoAreYou();    // "I am a cat"
```



```
A obj = new C();
obj.M();
```

10

10

HÀM TẠO VÀ THỪA KẾ

Gọi không tường minh tới hàm tạo của baseclass

```
class A {
  ...
}
```

```
class B : A {
  public B(int x) {...}
}
```

```
B b = new B(3);
```

Đúng

- A có hàm tạo mặc định A()
- B(int x)

```
class A {
  public A() {...}
}
```

```
class B : A {
  public B(int x) {...}
}
```

```
B b = new B(3);
```

Đúng

- A()
- B(int x)

```
class A {
  public A(int x) {...}
}
```

```
class B : A {
  public B(int x) {...}
}
```

```
B b = new B(3);
```

Lỗi

- Hàm tạo mặc định của A không tồn tại

Gọi tường minh

```
class A {
  public A(int x) {...}
}
```

```
class B : A {
  public B(int x)
  : base(x) {...}
}
```

```
B b = new B(3);
```

Đúng

- A(int x)
- B(int x)

11

11

PROTECTED VÀ INTERNAL

protected có thể được truy nhập trong cùng class và các subclass của nó

internal có thể được truy nhập trong cùng assembly

protected internal có thể được truy nhập trong cùng class và các subclass của nó, nhưng phải thuộc cùng một assembly

Ví dụ

```
class Stack {
  protected int[] values = new int[32];
  protected int top = -1;
  public void Push(int x) {...}
  public int Pop() {...}
}

class BetterStack : Stack {
  public bool Contains(int x) {
    foreach (int y in values) if (x == y) return true;
    return false;
  }
}

class Client {
  Stack s = new Stack();
  ... s.values[0] ... // lỗi khi dịch
}
```

12

12

LỚP TRỪU TƯỢNG

Ví dụ

```
abstract class Stream {
    public abstract void Write(char ch);
    public void WriteString(string s) { foreach (char ch in s) Write(ch); }
}

class File : Stream {
    public override void Write(char ch) {... ghi ch vào bộ nhớ ...}
}
```

Chú ý

- ❑ Các phương thức abstract không có phần cài đặt.
- ❑ Nếu một lớp có phương thức abstract (do khai báo hoặc do kế thừa) thì lớp đó phải là lớp trừu tượng.
- ❑ Không thể tạo ra đối tượng của lớp trừu tượng

13

13

THUỘC TÍNH VÀ INDEXER TRỪU TƯỢNG

Ví dụ

```
abstract class Sequence {
    public abstract void Add(object x); // phương thức
    public abstract string Name { get; } // thuộc tính
    public abstract object this [int i] { get; set; } // indexer
}

class List : Sequence {
    public override void Add(object x) {...}
    public override string Name { get {...} }
    public override object this [int i] { get {...} set {...} }
}
```

Chú ý

- ❑ Các thuộc tính và indexer được chồng phải có cùng các phương thức get/set tương tự như trong baseclass

14

14

LỚP CÔ LẬP (SEALED CLASS)

Ví dụ

```
sealed class Account : Asset {
    long val;
    public void Deposit (long x) { ... }
    public void Withdraw (long x) { ... }
    ...
}
```

Chú ý

- ❑ Không thể kế thừa từ *sealed* class, nhưng sealed class vẫn kế thừa từ lớp khác.
- ❑ Cũng có thể sử dụng từ khoá sealed với các phương thức

15

15

LỚP TỔNG QUAN (System.Object)

System.Object là baseclass của tất cả các lớp

```
class Object {
    protected object MemberwiseClone() {...}
    public Type GetType() {...}
    public virtual bool Equals (object o) {...}
    public virtual string ToString() {...}
    public virtual int GetHashCode() {...}
}
```

Sử dụng trực tiếp

```
Type t = x.GetType();    trả về kiểu được khai báo của x
object copy = x.MemberwiseClone();    tạo ra bản sao
```

Chồng trong subclass:

```
x.Equals(y)        so sánh giá trị của x và y
x.ToString()       trả về chuỗi biểu diễn x
int code = x.GetHashCode();    trả về mã băm của x
```

16

16

VÍ DỤ VỀ LỚP System.Object

```

class Fraction {
    int x, y;
    public Fraction(int x, int y) { this.x = x; this.y = y; }
    ...
    public override string ToString() { return String.Format("{0}/{1}", x, y); }
    public override bool Equals(object o) { Fraction f = (Fraction)o; return f.x == x && f.y == y; }
    }
    public override int GetHashCode() { return x ^ y; }
    public Fraction ShallowCopy() { return (Fraction) MemberwiseClone(); }
}

class Client {
    static void Main() {
        Fraction a = new Fraction(1, 2);
        Fraction b = new Fraction(1, 2);
        Fraction c = new Fraction(3, 4);

        Console.WriteLine(a.ToString()); // 1/2
        Console.WriteLine(a); // 1/2 (ToString được gọi tự động)

        Console.WriteLine(a.Equals(b)); // true
        Console.WriteLine(a == c); // false

        Console.WriteLine(a.GetHashCode()); // 3

        a = c.ShallowCopy();
        Console.WriteLine(a); // 3/4
    }
}

```

17